Music Distribution + Royalty Rail — MVP PRD

Product: Distribution-as-a-service for Ethiopian artists — get local music onto global DSPs, collect royalties back, split among contributors, pay out in birr.
Not in this MVP: streaming, video, fingerprinting, blockchain, AI features, publishing-royalty collection. Those are later phases and are deliberately cut.

I. The wedge

Ethiopian artists have no clean path to (a) global DSP distribution in their own language, (b) royalty collection back into ETB, and (c) fair, transparent splits among writers/producers/features. Streaming is a licensing + catalog + capital game we can't win as an entry point. Distribution + rights/splits is a real gap, small-team-buildable, and defensible (local payments, local languages, local trust). This product owns the artist-facing layer, local payments,

and the splits/royalty engine, and rides an existing distribution backend for the actual DSP pipes.

2. Hard dependency (highest risk)

We do not deliver directly to Spotify/Apple/etc. at launch. We integrate a white-label distribution backend (DaaS) that already holds DSP relationships and DDEX pipes. Candidates to diligence: FUGA, SonoSuite, Revelator, Believe/DPI-class backends. Before building anything, confirm: onboarding of an Ethiopian entity, ISRC/UPC provisioning, DDEX ERN ingest + DSR royalty reporting, per-release vs. revenue-share pricing. The schema is built so that if we later earn direct DSP access, only the delivery/reporting adapters change — the data model doesn't.

3. Users & roles

- Artist — self-releasing musician. Manages own artists, releases, splits, wallet, payouts.
- Label — one account managing many artists/releases; same tooling at scale.
- Contributor / payee — writer, producer, featured artist, financier. Holds splits and gets paid. May or may not be a platform account holder.

- Staff / Admin — internal review, delivery ops, royalty import, payout approval, support.

4. Core flow (end to end)

1. Onboard — sign up, create an artist profile, add payout method (Telebirr / Chapa / bank), KYC.
2. Create release — upload audio + 3000×3000 artwork, enter metadata (title, artists, genre, language, release date, territories), add tracks.
3. Set credits + splits — per track: credits (metadata for DSPs) and money splits (payees × %). Splits must total 100%.
4. Validate — automated rule engine flags errors (blocking) and warnings.
5. Pay fee — artist pays distribution fee (Chapa/Telebirr). (Your revenue.)
6. Submit → deliver — assign ISRC (per track) + UPC (per release), generate DDEX ERN, hand to DaaS, track delivery status per DSP until live.
7. Royalties in — DaaS/DSP reports (DDEX DSR) imported periodically; lines matched to tracks by ISRC.
8. Allocate — matched revenue split per track's split table → credit each payee's ledger (in USD).
9. Payout — payee requests or scheduled batch; convert USD→ETB at recorded rate; pay via

Telebirr/Chapa; debit ledger.

5. Functional requirements

5.1 Accounts & artists

- Email + auth (provider-agnostic; Firebase-compatible via `external_auth_id`).
- One user can own multiple artist profiles (label case).
- Artist profile stores DSP artist URIs (Spotify/Apple IDs) for correct delivery matching — critical to avoid songs landing on the wrong artist page.

5.2 Release & track ingestion

- Release types: single / EP / album / compilation.
- Track: title, ISRC (assigned at submit), audio asset, primary artist, explicit flag, language, preview start.
- Audio spec: WAV/FLAC/AIFF, ≥16-bit / ≥44.1 kHz. MP3 accepted but warned.
- Artwork spec: ≥3000×3000, RGB JPG/PNG, no URLs/social handles/pricing in image.

5.3 Metadata validation (blocking vs. warning)

Blocking errors (can't submit): missing audio, missing/undersized artwork, split total ≠ 100%, missing primary artist, release date in past for new release, missing genre/language. Warnings: MP3 source, no featured-artist credit but "feat." in title, very short/long tracks. Output stored as `validation_issues`.

5.4 Identifiers

- ISRC per track, UPC per release. Either allocated internally from a registrant prefix (via IFPI national agency) or provisioned by the DaaS — schema stores both source and value. Allocation is atomic (`identifier_counters`).

5.5 Distribution / delivery

- One `deliveries` row per release per DSP per action (deliver / update / takedown).
- Status machine: queued → sent → accepted → live / failed / taken_down, with an event log for debugging.
- Store DaaS message/release IDs and the live DSP URL.

5.6 Splits & contributors

- Credits (metadata shown on DSPs) and splits (money) are separate tables — a session

musician can be credited without a split; a financier can hold a split without a credit.

- Splits reference payees. Per (track, recording) splits ≤100% always (DB trigger), ==100% at submit (validation).
- Optional split-acceptance flow (invite payee to confirm their %) — status field present, enforcement is phase 2.

5.7 Royalty ingestion & accounting

- Import royalty batches (`royalty_reports`) → granular `royalty_lines` (ISRC, DSP, territory, quantity, revenue, currency, sale type).
- Match line → track by ISRC; unmatched lines queued for manual resolution.
- Matched line → `royalty_allocations` (per payee, per %) → `ledger_entries` credit. Full audit trail: line → allocation → ledger.

5.8 Wallet & payouts

- Balance = sum of ledger entries per payee per currency (append-only; no mutable balance).
- Accrue in report currency (USD). Payout converts USD→ETB at recorded `fx_rate` ; stores source amount, rate, and ETB paid.
- Methods: Telebirr, Chapa, CBE Birr, bank. Minimum threshold + hold period configurable

(`app_settings`).

- Payout lifecycle: requested → approved → processing → paid / failed; creates the debit ledger entry on success.

5.9 Billing (your revenue)

- `billing_charges` for the fees artists pay you to distribute (per-release fee and/or subscription), via Chapa/Telebirr. Kept separate from royalty ledger.

5.10 Admin

- Review queue, delivery ops, royalty import + match resolution, payout approval, KYC review, audit log of every state change.

6. Non-functional

- Languages (UI): Amharic, Afaan Oromo, Tigrinya, Somali, Afar, Sidama, English. Ethiopic + Latin script.
- Security: hashed/tokenized auth, 2FA for payout actions, least-privilege staff roles, full audit log, encrypted asset storage, signed upload URLs.
- Money correctness: all amounts as integer minor units (santim/cents) + ISO currency code. Never floats.

- Scale (MVP-realistic): thousands of artists, tens of thousands of tracks, monthly royalty batches. Design doesn't preclude growth; don't pre-build for 100M users.
- Availability: boring and reliable beats fancy. Managed Postgres, object storage for assets, background workers for delivery + royalty jobs.

7. Success metrics (first 6–12 months)

- Time from signup → first release live (target < 5 days incl. review).
- % releases delivered without manual metadata fixes.
- % royalty lines auto-matched by ISRC.
- Payout success rate + median payout time.
- Paying artists and gross distribution revenue (your fees).

8. Build sequence (inside the MVP)

1. Accounts + artist profiles + payout methods.
2. Release/track ingestion + asset upload + validation engine.
3. Identifier allocation + DDEX export + DaaS delivery adapter (deliver/takedown + status).

4. Splits + payees + credits.
5. Royalty import + matching + allocation + ledger.
6. Wallet + payouts (Telebirr/Chapa + FX).
7. Billing (Chapa fee collection).
8. Admin console.

9. Open decisions (yours to make)

1. DaaS vendor — the gating dependency; validate onboarding + terms first.
2. Fee model — per-release ETB fee vs. subscription vs. revenue share (or hybrid).
3. FX source — which USD→ETB rate for payouts (NBE reference, bank, or a spread you set), given the floated birr.
4. Minimum payout threshold + hold period — protects against clawbacks/chargebacks in DSP reporting.
5. ISRC source — obtain own registrant prefix vs. use DaaS-provisioned.

Companion file: `distribution-royalty-schema.sql` — production-oriented Postgres DDL implementing everything above.